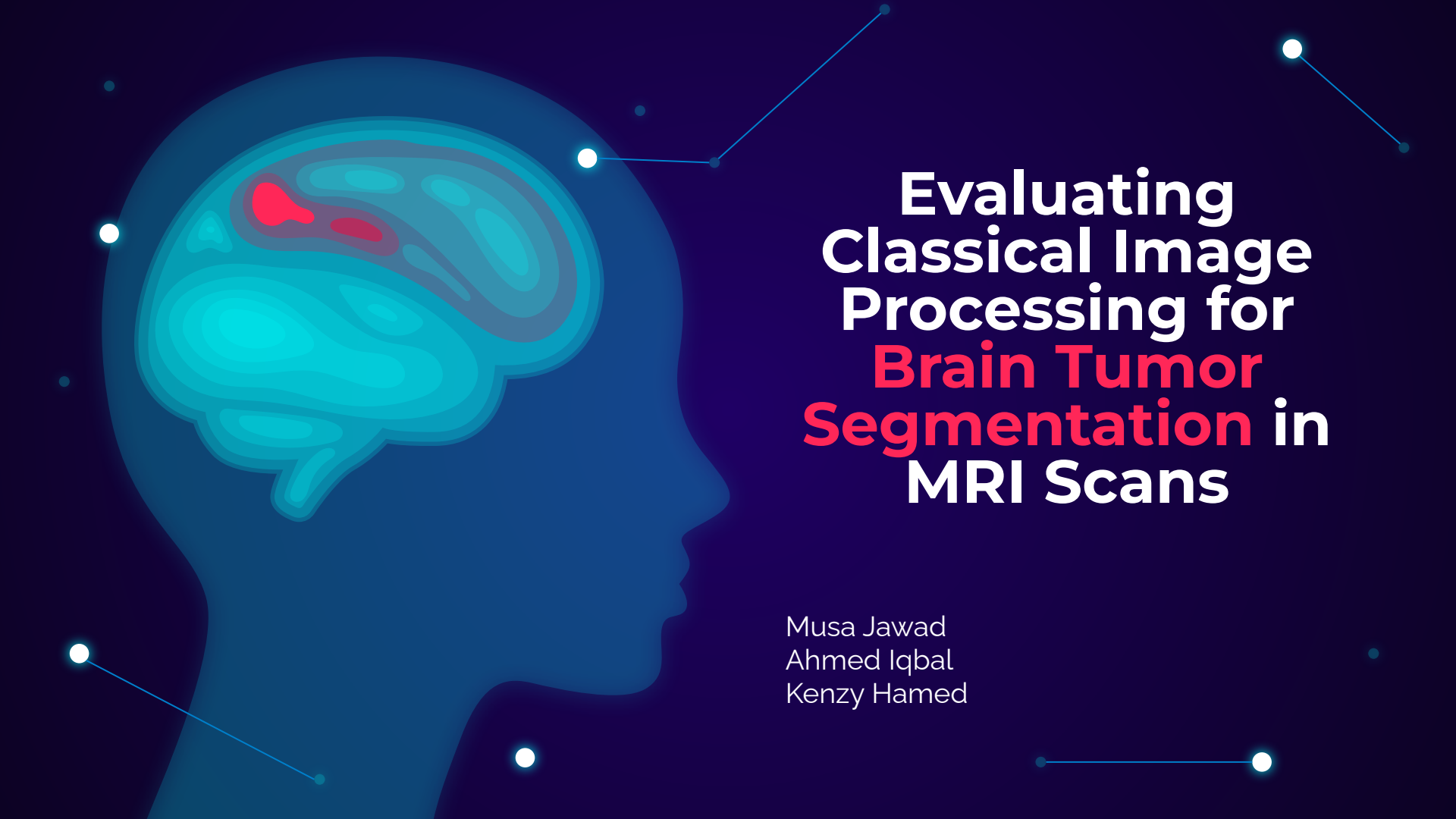
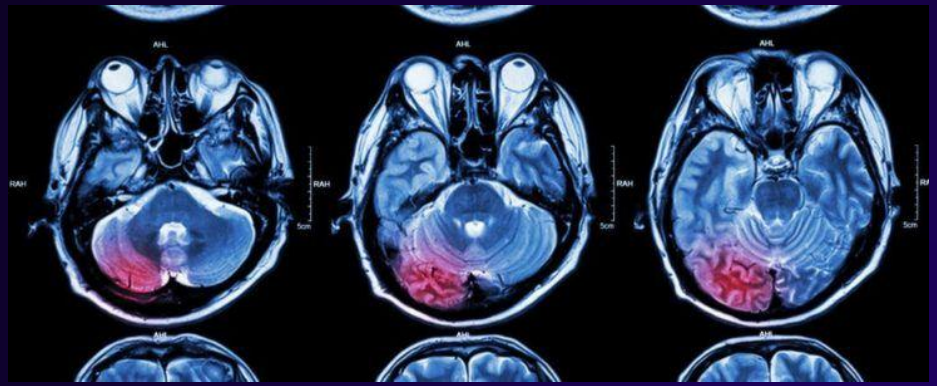


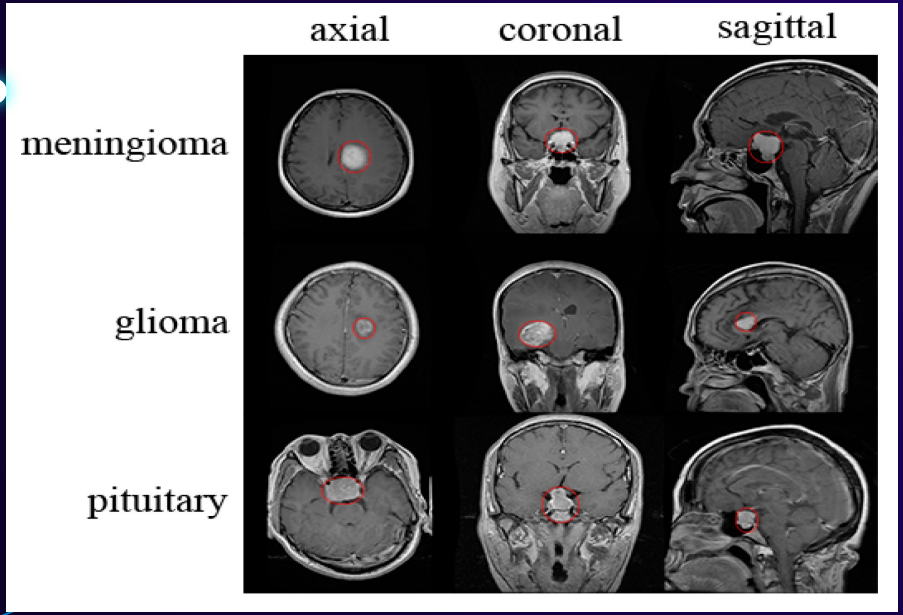
Evaluating Classical Image Processing for Brain Tumor Segmentation in MRI Scans

Musa Jawad
Ahmed Iqbal
Kenzy Hamed



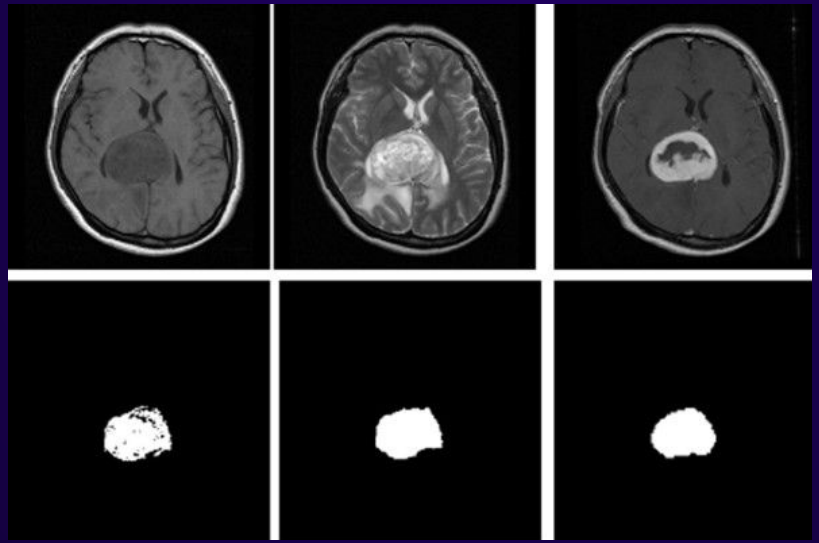
What are Brain Tumors

- A tumor is an abnormal growth of cells within your body. When this growth occur in the brain, it is called a intracranial tumor, aka brain tumor.
- They are caused by malfunction in DNA which cause the cells to continue living, when otherwise a healthy cell would die.
- There are many different types of brain tumors which all depend on the cells from which the tumor grows from. Some types are:
  - Gliomas
  - Meningiomas
  - Pituitary Tumor



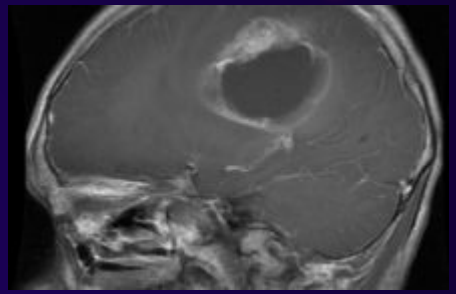
Our Objective

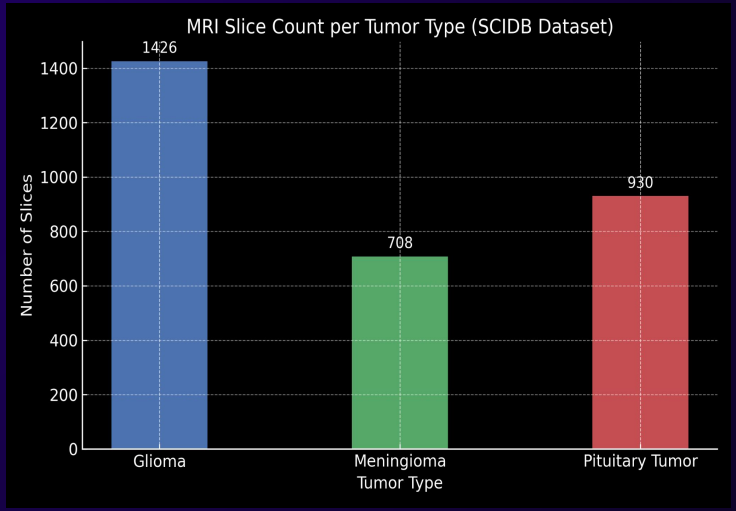
Our objective is to develop an image processing pipeline which will segment brain tumors from MRI scans.



Tools & Materials

- Dataset Selection
  - Initially planned to used CBICA (Center for Biomedical Image Computing & Analytics) dataset
  - Instead, we selected a SCIDB Brain Tumour Dataset, better suiting our project
    - Contains 3000+ grayscale MRI images from 233 patients (Glioma, Meningioma, Pituitary)
- Frameworks
  - Python, OpenCV, Tensorflow, Matplotlib, NumPy, scikit-learn





Method 1- Otsu's Thresholding

- Simple approach
  - No training required
  - Good baseline for tumour segmentation
- Method
  - Otsu's method finds a global threshold, and separates the image to:
    - Bright areas (tumor)
    - Darker areas (brain tissue)
- Results
  - Led to over-segmentation and low Dice scores, but helped us confirm our pipeline

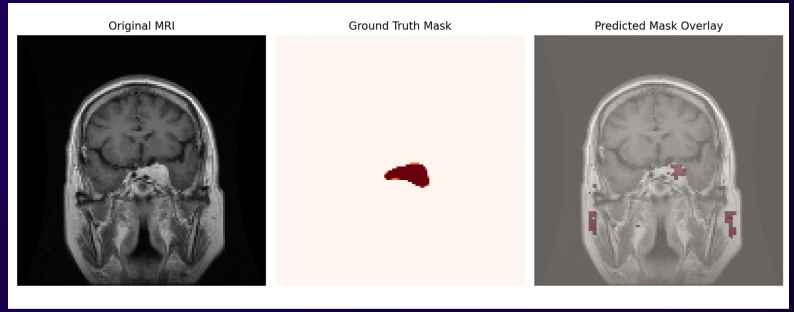
Method 2: U-Net model

```
Loaded 3064 MRI scans.
Epoch 1/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 12s 20ms/step - accuracy: 0.9668 - loss: 0.1714 - val_accuracy: 0.9822 - val_loss: 0.0717
Epoch 2/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 13s 21ms/step - accuracy: 0.9824 - loss: 0.0712 - val_accuracy: 0.9822 - val_loss: 0.0721
Epoch 3/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 22ms/step - accuracy: 0.9831 - loss: 0.0689 - val_accuracy: 0.9822 - val_loss: 0.0705
Epoch 4/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 22ms/step - accuracy: 0.9823 - loss: 0.0699 - val_accuracy: 0.9822 - val_loss: 0.0667
Epoch 5/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 23ms/step - accuracy: 0.9822 - loss: 0.0679 - val_accuracy: 0.9822 - val_loss: 0.0649
Epoch 6/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 23ms/step - accuracy: 0.9827 - loss: 0.0649 - val_accuracy: 0.9822 - val_loss: 0.0661
Epoch 7/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 23ms/step - accuracy: 0.9823 - loss: 0.0655 - val_accuracy: 0.9822 - val_loss: 0.0665
Epoch 8/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 23ms/step - accuracy: 0.9827 - loss: 0.0643 - val_accuracy: 0.9822 - val_loss: 0.0629
Epoch 9/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 23ms/step - accuracy: 0.9828 - loss: 0.0635 - val_accuracy: 0.9822 - val_loss: 0.0625
Epoch 10/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 23ms/step - accuracy: 0.9824 - loss: 0.0640 - val_accuracy: 0.9823 - val_loss: 0.0633
Epoch 11/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 23ms/step - accuracy: 0.9823 - loss: 0.0643 - val_accuracy: 0.9825 - val_loss: 0.0607
Epoch 12/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 23ms/step - accuracy: 0.9829 - loss: 0.0618 - val_accuracy: 0.9826 - val_loss: 0.0615
Epoch 13/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 15s 24ms/step - accuracy: 0.9830 - loss: 0.0614 - val_accuracy: 0.9824 - val_loss: 0.0603
Epoch 14/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 15s 25ms/step - accuracy: 0.9831 - loss: 0.0609 - val_accuracy: 0.9826 - val_loss: 0.0626
Epoch 15/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 15s 25ms/step - accuracy: 0.9826 - loss: 0.0625 - val_accuracy: 0.9824 - val_loss: 0.0604
Epoch 16/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 15s 25ms/step - accuracy: 0.9831 - loss: 0.0607 - val_accuracy: 0.9829 - val_loss: 0.0596
Epoch 17/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 14s 22ms/step - accuracy: 0.9825 - loss: 0.0620 - val_accuracy: 0.9830 - val_loss: 0.0598
Epoch 18/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 13s 22ms/step - accuracy: 0.9826 - loss: 0.0616 - val_accuracy: 0.9829 - val_loss: 0.0587
Epoch 19/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 16s 26ms/step - accuracy: 0.9836 - loss: 0.0582 - val_accuracy: 0.9827 - val_loss: 0.0618
Epoch 20/20
613/613 ━━━━━━━━━━━━━━━━━━━━ 13s 21ms/step - accuracy: 0.9823 - loss: 0.0623 - val_accuracy: 0.9827 - val_loss: 0.0608
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step
Dice Score (Sample 0): 0.2734
2025-03-24 21:50:21.613 python[45863:4715264] +[IMKClient subclass]: chose IMKClient_Modern
2025-03-24 21:50:21.613 python[45863:4715264] +[IMKInputSession subclass]: chose IMKInputSession_Modern
```

INPUT (128x128x1)
- Conv2D (32 filters) → MaxPooling
  - Conv2D (64 filters)
    - Conv2DTranspose (upsample)
      - Concatenate with earlier conv
        - Conv2D (1x1, sigmoid output)

- Training and validation accuracy (~98%)
- Dice Similarity Coefficient, we obtained a score of 0.2734

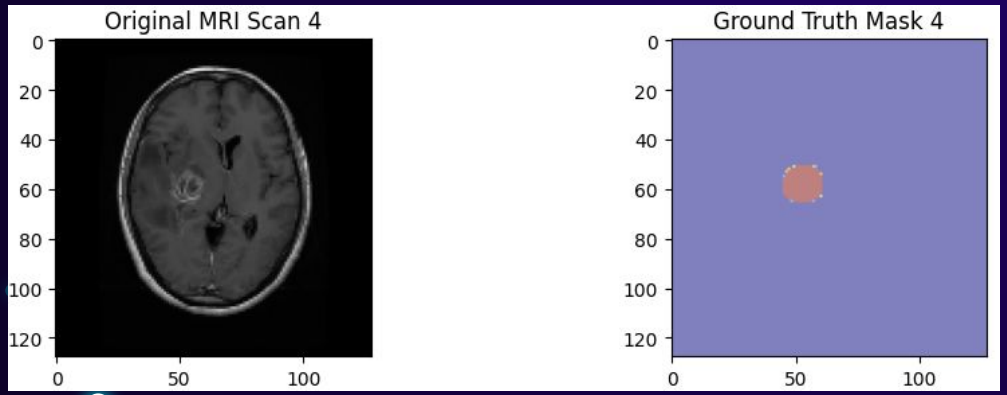
```python
def unet_model(input_size=(128, 128, 1)):
    inputs = Input(input_size)

    # Encoder
    conv1 = Conv2D(32, 3, activation='relu', padding='same')(inputs)
    pool1 = MaxPooling2D()(conv1)

    conv2 = Conv2D(64, 3, activation='relu', padding='same')(pool1)

    # Decoder
    up1 = Conv2DTranspose(32, 2, strides=2, padding='same')(conv2)
    merge1 = concatenate([up1, conv1])
    conv3 = Conv2D(1, 1, activation='sigmoid')(merge1)

    model = Model(inputs=inputs, outputs=conv3)
    model.compile(optimizer=Adam(), loss='binary_crossentropy', metrics=['accuracy'])
    return model
```

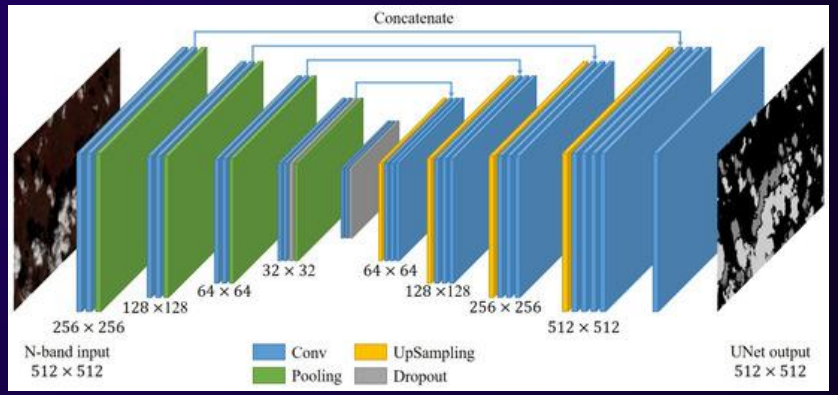


Method 3 - Final Approach

```python
def build_model(input_shape=(128, 128, 1)):
    model = Sequential([
        Conv2D(64, (3, 3), activation='relu', padding='same', input_shape=input_shape),
        MaxPooling2D((2, 2)),
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        MaxPooling2D((2, 2)),
        Conv2D(256, (3, 3), activation='relu', padding='same'),
        UpSampling2D((2, 2)),
        Conv2D(128, (3, 3), activation='relu', padding='same'),
        UpSampling2D((2, 2)),
        Conv2D(64, (3, 3), activation='relu', padding='same'),
        Conv2D(1, (1, 1), activation='sigmoid')
    ])
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model
```

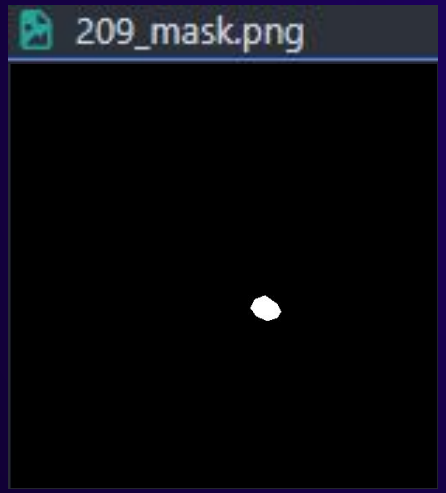
- Turn the images greyscale and create a binary mask of it. The pixel = 1 if there is tumor, or 0 otherwise.
- 128 by 128 mask
- U-net Convolutional Neural Network
  - Made in 2015
  - Used for upscaling images, creating masks for images, and generating images
  - Has 2 parts
    - Encoder: Captures the features of the time of the image
    - Decoder: Captures where the item is
    - Combining these 2 can give us a pixel perfect mask of the original object

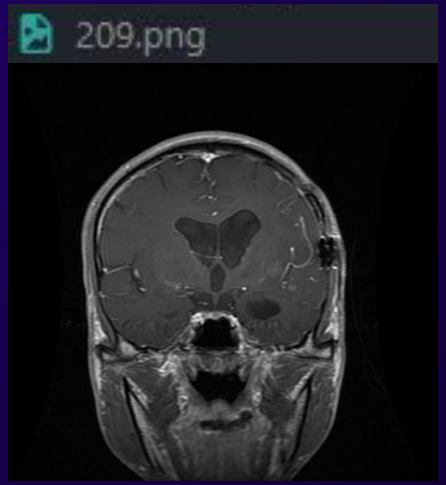




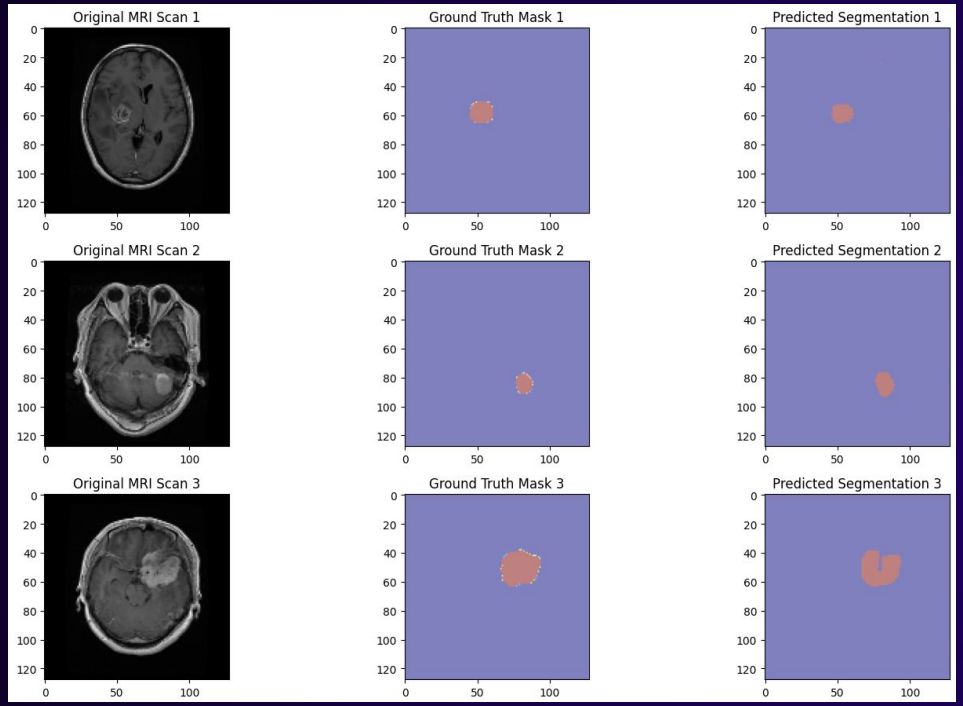
Method 3 - Final Approach - cont.

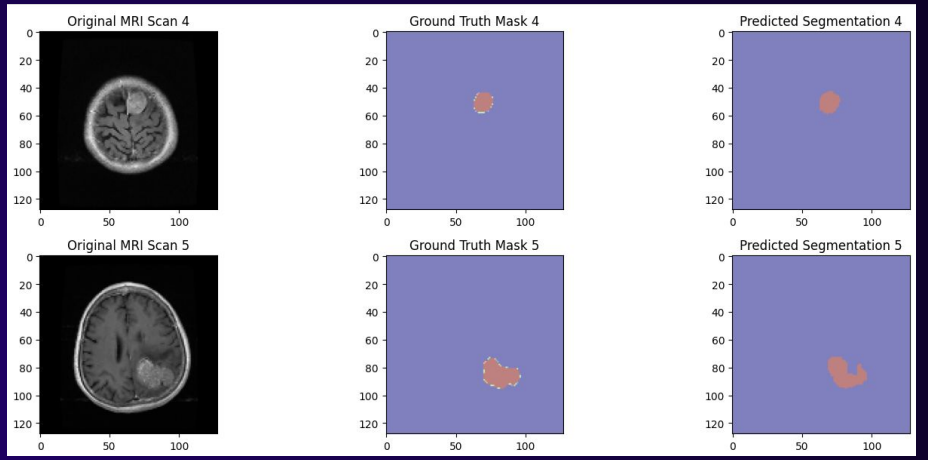
- Resize each MRI image to a 128 by 128 pixel image
- Apply grayscale to each image
- Insert image into the U-net model to predict the mask, and then compare to the already given truth mask





Our Results

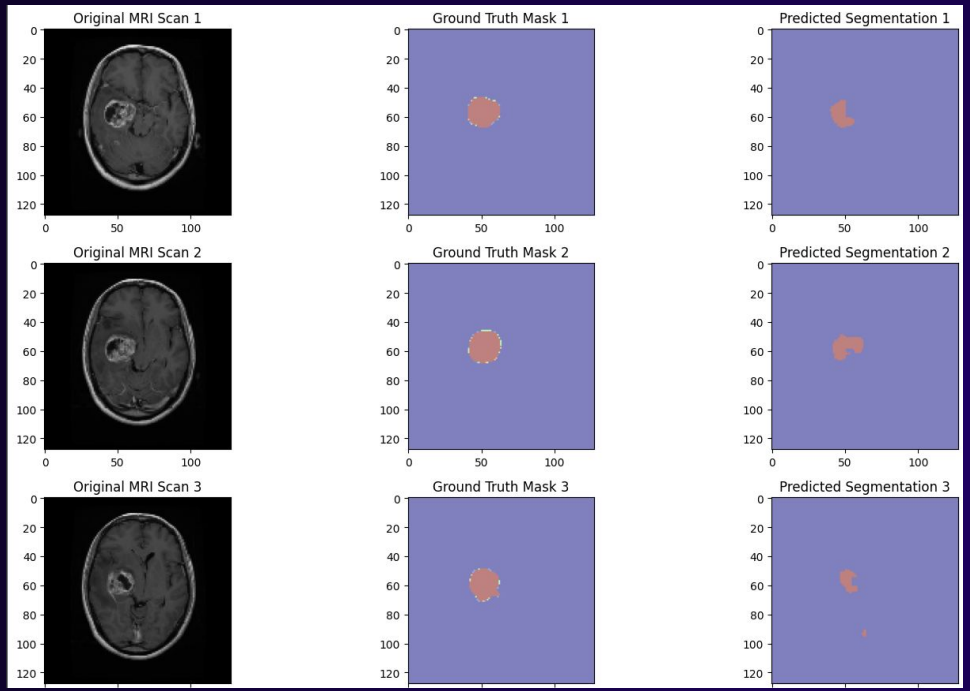


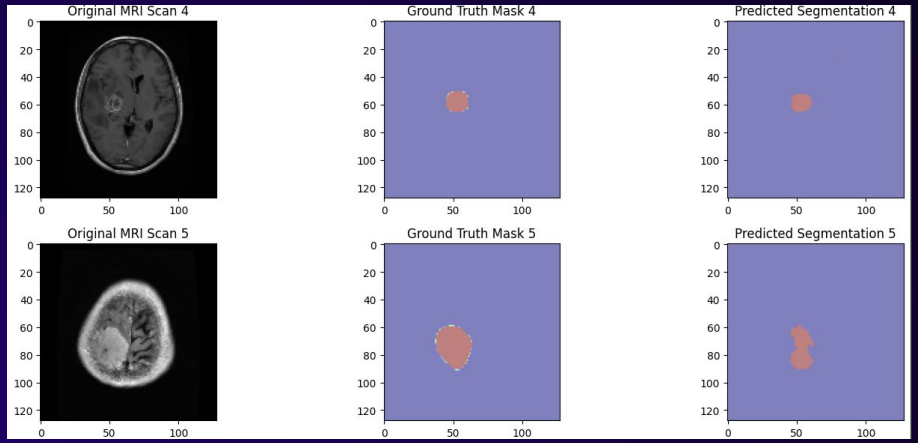


Metrics

Example	DSC	IoU	Accuracy
1	0.8700	0.7699	0.9967
2	0.8656	0.7631	0.9971
3	0.9159	0.8449	0.9939
4	0.9328	0.8741	0.9980
5	0.8620	0.7575	0.9924

Our Results - Limitations





Metrics

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 108ms/step
Example 1: DSC = 0.6500, IoU = 0.4815
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
Example 2: DSC = 0.7501, IoU = 0.6001
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step
Example 3: DSC = 0.5072, IoU = 0.3398
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
Example 4: DSC = 0.8700, IoU = 0.7699
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
Example 5: DSC = 0.7171, IoU = 0.5589
```

Conclusion & Future Work

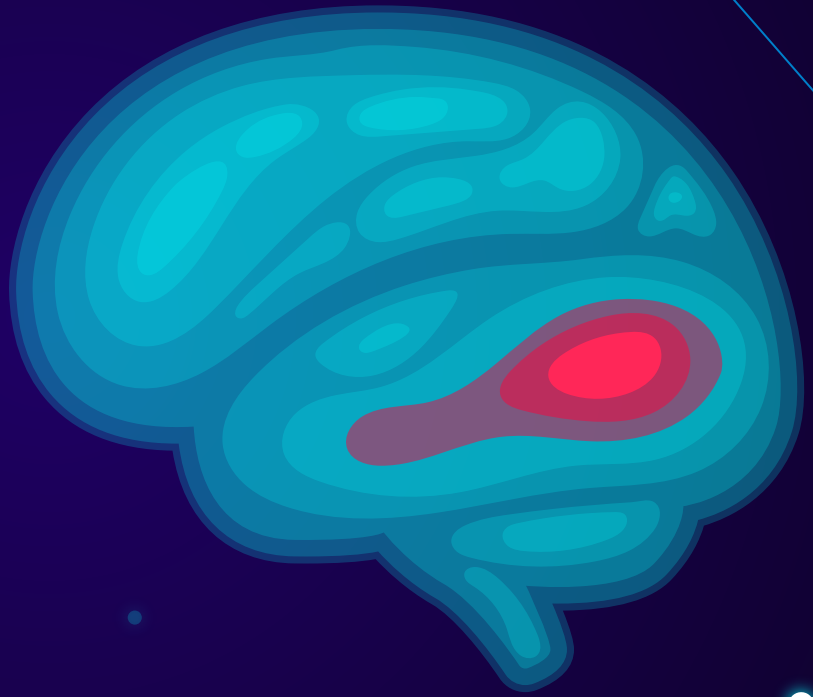
Conclusion:

- Otsu's method wan't well suited, due to its reliance on global threshold
- Our first U-net model provided high accuracy but did not show the same results for dice score
- The final U-net model was able to accurately learn tumour features/locations, giving us better metrics

Future:

- Increase IOU Score
- Determine if there is tumor to begin with rather than always assuming there is
- Model Deployment

Thanks



CREDITS: This presentation template was created by **Slidesgo,** including icons by **Flaticon,** and infographics & images by **Freepik**